UAS SMS OTP Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS SMS OTP Implementation Guide

This document provides the concept, implementation steps, and API integration details of the i-Sprint Short Message Service (SMS) One-Time Password (OTP) with AccessMatrix Universal Authentication Server (UAS) solution.

Intended Reader

It is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers to carry out the following operations:

- Integrate i-Sprint SMS OTP with AccessMatrix server for authentication purposes.
- Configure Delivery Channel to support OTP delivery via SMS or email.
- Implement SMS OTP authentication using API.

Document Scope

This document only provides the necessary steps to configure and set up i-Sprint SMS OTP. It will not provide details on how the SMS/Email OTP is generated and verified in the AccessMatrix system.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Common Administration Guide*
- *AccessMatrix REST API Specification*

2. Introduction to UAS SMS OTP Implementation

This document provides the instructions on the configuration and API integration of i-Sprint SMS OTP.

About UAS SMS OTP

UAS SMS OTP provides a secure authentication solution by using two separate channels to authenticate a user. This solution delivers an OTP to users via GSM Short Message Service (SMS) or email. For example, when a user tries to access a website, an OTP will be sent to his mobile phone via SMS or email (determined by the user in advance). In addition, the mobile devices can now become security tokens to receive OTPs, which can strengthen the existing Id/Password authentication and authorization process.

The complexity (length and format) and the expiration of the generated OTP can be configured to comply with the organization's security policies.

Key Benefits

- Robust OTP authentication solution via the AccessMatrix UAS flexible integration framework and granular security policies.
- Enhance security with SMS OTP authentication to mitigate phishing attacks.
- Leverage HSM hardware to provide advanced OTP generation and comparison options.
- Maximize ROI by reducing implementation costs and project risks by deploying a proven security solution with solid track records.
- Ensure compliance with powerful reporting capabilities to report user activities and security violations.

Features

- **Flexible OTP Policies**
    - Expiry time
    - OTP format (length, number, alpha, alphanumeric)
    - Outgoing message template
    - One-time use vs Multi-use
    - Restriction to number of retries
    - Personal Authentication Code (PAC) to improve user experience
- **Audit Trail**

- UAS i-Sprint SMS OTP provides comprehensive tamper-evident audit trail information to track OTP usage and address the transaction audit requirements. In addition, it offers flexible reporting capabilities for administration, user activities and security violations.

- **Multiple Delivery Mechanisms**
  - UAS supports the delivery of i-Sprint SMS OTP via SMTP, SMPP, HTTP/S POST/GET, Web Service based APIs.
  - Flexible Integration SDKs
  - UAS provides comprehensive SDKs with REST APIs to various kinds of applications to be easily integrated.
- **Built-in RADIUS Server**
  - The built-in server offers RADIUS Server for solid authentication to firewalls, network devices, VPN servers or any server platforms and applications that support the RADIUS authentication protocol.
- **Native Integration**
  - Security Server supports several user registries such as LDAP and Active Directory as external user stores via LDAP protocol or JDBC.
  - Organizations can integrate the Security Server with their existing user registries without the need to synchronize user information.
  - UAS can access the external user store(s) directory to simplify the integration efforts. No schema change is required, and no information needs to be written to the external user stores.
- **Key Management & Crypto Processing**
  - UAS has proven integration with HSM devices to protect the master key and support OTP generation and comparison inside the HSM for high-security needs.
- **Operational Standard**
  - UAS provides proven scalability and reliability features to meet the most stringent service level and operational requirements for large scale deployments.

i-Sprint SMS OTP Token Life Cycle

The UAS SMS OTP solution has the following life cycle:

- **SMS OTP Generation**

- **SMS OTP Delivery**
- **SMS OTP Validation**
- **SMS OTP Housekeeping**

SMS OTP Generation

An SMS OTP token is a virtual token that can have a lifespan ranging from a few minutes to a few days. A new SMS OTP token is created when the application calls the generateOTP API with a new token UUID (Universal Unique Identifier) which will never change once created. If PAC is enabled in the SMS OTP token policy, the newly generated token will contain a new OTP and PAC, and the OTP hash will be stored as a token property within the token. PAC is the random string sent back to the user to associate with the OTP.

SMS OTP token records are saved in:

1. **Memory** - This is the default setting where the token store is specified as MemoryTokenStore.
2. **Database** - If an application calls API to specify the token store as SystemTokenStore.

If MemoryTokenStore is used and the application expects generation of OTP in AccessMatrix server instance 1 and verification of OTP on AccessMatrix server instance 2, then EhCache replication of MemoryTokenStore must be set up correctly.

By default, when an SMS OTP token is created, its status is "Activated".

SMS OTP Delivery

SMS OTP delivery is defined in the delivery channel associated with an event notification policy which is configured to use a specific message template. The token can be delivered either via SMS or Email. Refer to Setup Steps for details.

SMS OTP Validation

When the token is correctly verified, its use count is incremented; otherwise, the bad login count is incremented until it is locked.

OTP Validation may result in a status change:

- **Expired** - When an attempt to verify OTP after token exceeds its timeout value as set up in SMS OTP token policy.

- **Locked** - When maximum failed attempts are reached as configured in SMS OTP token policy.
- **Invalidated** - When a code replay attempt is detected where the token policy does not allow reuse of OTP.

Whether OTP is verified successfully or not, SMS OTP tokens that exceed the validity timeout period will be deleted (either from the memory or from the database).

SMS OTP Housekeeping

By default, the housekeeping policy must be configured to purge away obsolete tokens for the usage of `SystemTokenStore`. The default setting is three days. For high traffic websites, this can be reduced to one day. Refer to SMS Token Housekeeping Policy for more details.

Client applications can also explicitly delete the token by calling the relevant method `deleteToken` in `TokenService` when necessary. Deleting SMS OTP Token requires an authentication session that has a token admin role (e.g. "TokenAdminRole"). Use the admin console to grant "TokenAdminRole" to the authenticated session user before the token can be deleted.

Implementation Checklist

Below is the checklist for implementing the UAS SMS OTP solution.

Tasks	References
Configure the SMS OTP Token Policy	SMS OTP Token Policy
Configure the Message Template	Message Template
Configure the Message Delivery Channel	Delivery Channel
Configure the Event Notification Policy	Event Notification Policy
Configure SMS OTP Login Module	SMS OTP Login Module
Configure the Authentication Realm	Authentication Realm
Configure User Accounts	User Accounts

Implementation Planning

Planning prior to environment setup is critical to the success of UAS SMS OTP implementation. Understanding the customer's nature of business, studying the customer's requirements and its existing environments will help to identify the actions to make prior to actual implementation.

The following sections discuss the implementation considerations that should be assessed during planning.

Security Consideration

The generation of SMS OTP will be according to a configurable SMS OTP token policy. The token policy governs how the token used for authentication or authorization behaves. Therefore, the admin must first identify the token policy to be used and whether the application needs to supply extra parameters for the generation of OTP.

The generation of OTP is based on:

- SMS OTP token policy used (use the admin console to set the length, format of OTP and Personal Assurance Code (PAC)),
- Java SecureRandom generated a random number, and
- (Optional) User, application, transaction and session-related parameters supplied by the application

The SMS OTP Token policy also allows the administrator to control:

- Whether OTP can be verified correctly more than once
- The maximum number of attempts to verify an OTP before locking the token
- OTP length
- OTP expiry

For SMS OTP API, it is possible to bind the OTP with additional verification information to allow verification information (e.g. session ID, transaction ID, device ID) to be provided as part of the OTP generation call. However, the same information needs to be provided during the verification API call to verify the OTP. This information is provided by the application (not the user).

Integration Consideration

This section discusses the SMS OTP integration's requirements. It is essential to identify how i-Sprint SMS OTP solution will be used.

- **Identify the use case**

  Determines if SMS OTP will be used for user authentication at the main entry point (login flow)(For example, login flow will ask for the user's login ID and 1FA/password, followed by 2FA/SMS OTP. SMS OTP can proceed and be considered a logged-in user when the user has been successfully authenticated using both passwords.) Determine if SMS OTP will be requested only when the user is trying to access a more sensitive page (with elevated authentication)(For example, login flow will only ask for 1FA/password. After 1FA, the user is considered logged in and can access basic information (e.g. account balance). However, when the user tries to access more sensitive information, such as detailed account information, 2FA/SMS OTP will be required. After the user login session has been elevated, subsequent access to sensitive information will no longer challenge the user with SMS OTP.) Determine if SMS OTP will be used during critical transaction points such as:

  - Changing of user ID or email address
  - Setting up of funds transfer target
  - Performing the high-value transaction

- **Identify whether a user is searchable from UAS server**

  For UAS server to evaluate the message template and send out the SMS or email, certain user information must be obtained either from an existing user store or supplied by the application during generating OTP call:

  - If the user is not searchable from UAS server, the application must provide a mobile phone number or email address during the generation of OTP.
  - If the user is searchable, email address or mobile phone number or other user information can be read from default user store or external user stores like Active Directory based on user store attribute mapping to be used within the message template. User stores must be set up using the web admin console first.

- **Identify what to display to the user**

  SMS OTP module requires two steps: to generate and send OTP and to verify OTP. As part of OTP generation, a PAC (normally a four-character code) is returned to the API caller to be displayed to the user when asking for the OTP. The OTP page needs to determine whether to

display this PAC. The OTP page also needs to identify what to show to the user for each type of potential error (e.g. token expired, max failed attempt reached, token locked, and code replay attempt).

Performance Consideration

Storage

By default, AccessMatrix UAS stores the SMS OTP session in memory (MemoryTokenStore). This is to allow faster performance and less maintenance as it does not require database housekeeping. For this setup, it is important to configure cache replication so that the OTP session can be replicated to other instances. The cache replication is normally done in an asynchronous manner; thus, there is a short delay of when can the OTP be verified in other instances. For SMS OTP scenario, normally, this should not be a problem as the sending of the OTP to the user will take at least a few seconds.

AccessMatrix UAS can also persist the OTP session into the database (SystemTokenStore) if specified during the API call. This will eliminate the need for cache replication. However, this will incur some costs in retrieving and updating the information in the database. There will also be a need to configure a housekeeping task to housekeep the expired or used OTP sessions. Persisting in a database is normally required only when the OTP needs to be valid for a very long time (e.g. days). For most cases, storing in memory (MemoryTokenStore) is recommended.

Delivery Queue

During OTP generation, the OTP will be delivered to the user by SMS or email. In AccessMatrix UAS, OTP delivery is done asynchronously for performance reasons (so that the API call can return and get the OTP generation response immediately without waiting for the message gateway). The message delivery framework has a message delivery queue and worker threads (to process the delivery). The default configuration has been configured to fit the SMS OTP use case. However, if the load is much higher or if the message gateway is not fast enough, there is a need to tune the message delivery queue and worker threads further.

- If the message queue gets more than half the load often and yet the message gateway can actually handle more load, then it is recommended to increase the number of worker threads.
- If there is a scenario where suddenly there is a burst of OTP requests and the message gateway is not slow, then it is possible to increase the queue size to make space for the burst requests

and let the message gateway clear the requests. However, if the OTP expiry is very short, then this may not be an option.

- If the message gateway is slow to respond, the best solution is to contact the message gateway vendor to speed up the process. Increasing the worker thread and queue will not help or it may make the problem worse.

The worker thread and delivery queue set are managed per Delivery Channel module. That means, if there are two delivery channels configured in AccessMatrix UAS (e.g. SMS for Internal and SMS for External), then each will have its own queue and worker threads. This is to increase reliability so that a slowdown in one channel does not affect the other channel (provided they are using a different message gateway).

In the case when the message delivery queue is full, it will wait only for one second (default) to wait for a space in the queue. If the queue is still full, then it will reject the request. This is done to increase reliability not to further overload as SMS OTP is normally time-sensitive. It is not useful to wait for a long time to send the OTP when the OTP has already expired.

3. UAS SMS OTP Configuration

This section defines the steps relating to the implementation of UAS SMS OTP. Before you start the implementation, it is assumed that you have completed the planning and implementation checklists and you have understood the concept of the UAS SMS OTP solution.

- **Configure the message template**

  Message Template is used to create standard messages that allow you to send messages with similar content. A part of the content is predetermined, while some content can be added when sending the message. It is one of the two main components used for event notification policy setup. A new template can be created; however, predefined message templates can be utilized by applying relevant changes to the message templates, if necessary.

  Refer to Configure Message Template for details.

- **Configure the delivery channel**

  Delivery Channel defines the channel to deliver the OTP message to the target recipient, e.g. via the SMS gateway server. A new delivery channel can be created; however, predefined delivery channels can be utilized by applying relevant changes to the delivery channels, if necessary.

  Refer to Configure Delivery Channel for details.

- **Configure the event notification policy**

  Event Notification Policy completes the entire process of message administration after both message templates and delivery channels are configured. It specifies the event that will trigger the OTP generation and delivery based on the associated Message Template and Delivery Channel.

  Refer to Configure Event Notification Policy for details.

- **Configure the SMS OTP token policy**

  SMS OTP Token Policy manages the behavior of the SMS OTP used for authentication, e.g. OTP length and OTP validity period. This policy is to be used during login module administration. By default, a predefined SMS OTP policy called "DefaultSmsOtpPolicy" will be created upon successful installation of AccessMatrix. However, a new SMS OTP token policy can also be created.

  Refer to Configure SMS OTP Token Policy for configuration details.

- **Configure the i-Sprint SMS OTP login module**

  i-Sprint SMS OTP login module provides an i-Sprint SMS OTP login method for users who may reside in either an internal or external user store. By default, a predefined i-Sprint SMS OTP login module called "SMSOTPLogin" will be created upon successful installation of AccessMatrix.

However, a new i-Sprint SMS OTP login module can also be created.

Refer to Configure SMS OTP Login Module for configuration details.

- **Configure the Authentication Realm to assign to the user**

  The Authentication Realm associates users' credentials for authentication based on a lookup module and login module(s). User entries can be created into the local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). By default, a predefined i-Sprint SMS OTP authentication realm called "ADSMS" will be created upon successful installation of AccessMatrix. However, a new authentication realm can also be created.

  Refer to Configure Authentication Realm for configuration details.

  > **Note**: It is assumed that the lookup module is already configured and pointing to the user store. Refer to *AccessMatrix Common Administration Guide - Configure User Lookup Module* for configuration details. Depending on the use case requirements, another login module may be created and assigned to the same authentication realm.

- **Assign login accounts to the user**

  User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the "Login Accounts" tab.

  Refer to Configure User Accounts for details.

- **Configure housekeeping policy**

  Refer to SMS Token Housekeeping Policy for configuration details.

- **Configure SMS OTP logging**

  Refer to B. SMS OTP Logging for details.

3.1. Configure Message Template

To create a new message template, navigate to **Administration Dashboard > Configuration > Messaging & Response** and click **Message Template** on the left pane. Next, click **Create** and select "Default Template Engine" from the **Choose Message Template Implementation** page. Then, click **Next** and provide the details for the message template attributes before saving the changes.

Attribute Name	Description
*Message Template Id	Unique Id of message template.
*Message Template Name	Name of message template.
Description	Description of message template.
Implementation	Type of message template implementation, i.e. "Default Template Engine". (uneditable)
Configuration Tab	
Sender and Recipients	
Note: Macros can be used for the sender and recipients information. Refer to Predefined Macros - Sender and Recipients for the list of predefined macros. However, you may also enter the actual address directly, such as an email address or mobile number, as long as it is supported by the delivery channel (see Delivery Channel).	
*From	Sender of the message.
*To	Primary recipient(s) of the message. **Note:** This can be an email address or mobile number.
Cc	List of the recipient(s) under CC list. **Note:** This can be an email address or mobile number.
Bcc	List of recipient(s) under BCC list. **Note:** This can be an email address or mobile number.
Message Body	
Note: Macros can be used for the message body. Refer to Predefined Macros - Body for the list of predefined macros.	
Subject	Refer to the summary of the message.

Attribute Name	Description
Body	It contains the message body's content.
Allow File Attachment	Enabling this feature will allow files to be attached in the message template. Default: false
Escape HTML	If set to "true", HTML escape will be performed on the message content. Enable this only when the intended message content is HTML. Default: false

Predefined Macros

Below is the list of commonly used macros. To view the complete list of predefined macros, click the **Help** icon found in the **Message Template's Configuration** tab in the admin console.

Macro Name	Replaced by
Sender and Recipients	
${userId}	User Id
${userName}	User name
${userEmail}	User's email
${userMobile}	User's mobile phone number
${targetUserEmail}	Target user's email
${targetUserMobile}	Target user's mobile
$userAttr.get("<name>")	Any user attribute name
Body	
${otp}	One-Time Password (OTP)
${pac}	Personal Assurance Code (PAC)
${issuedate}	Issue date of the SMS OTP Token.
${issuetime}	Issue time of the SMS OTP Token.
${issueDateObj}	Issue date of the SMS OTP Token.

Macro Name	Replaced by
${expiredDateObj}	Expiration date of the SMS OTP Token.
${timeout}	Expiry timeout in seconds as specified in SMS OTP token policy.
${event.<name>}	A special attribute passed by client API.

Notes:

- The user may specify the key name without the curly brackets '{...}', e.g. $userId. The curly brackets should be used to avoid ambiguousness whenever necessary.
- For date-related macros, the formatting of the date can be changed. To do so, use the macro "${dateTool.format('<desired format>', <date-related macro>)}". For example, using the macro "${dateTool.format('dd/MM/yyyy hh:mm:ss', ${expiredDateObj})}" will produce the expiration date of the SMS OTP token in the format "04/10/2023 12:37:15". You can refer to the dateTool symbol table in *AccessMatrix Common Administration Guide - MessageTemplate Macros* for a detailed explanation of the formatting options.

Usage Examples

Example 1:

```
Your OTP is $otp, PAC is $pac, valid for $timeout seconds issued at $issuetime on $issuedate )
```

User will receive the following sample message:

```
Your OTP is 95159731, PAC is 5515, valid for 900 seconds issued at 3:57:34 PM on Mar 31, 2010
```

Example 2:

To display $timeout in minutes instead of seconds:

```
#set( $Integer = 0)
#set ($timeoutmin = ${Integer.parseInt($timeout)} / 60)
Your OTP is $otp, PAC is $pac, valid for ${timeoutmin} minutes issued at $issuetime on $issuedate
```

User will receive the following sample message:

```
Your OTP is 93858246, PAC is 7432, valid for 15 minutes issued at 3:57:34 PM on Mar 31, 2010
```

Example 3:

To split OTP into two parts, two message templates and two delivery channels should be created to handle the 1st and 2nd half of OTP. Both sets of message templates and delivery channels should be configured into the same event notification policy Id.

First message template:

```
#set ($val1= $otp.substring(0, 3))
Your OTP is $otp, PAC is $pac, valid for $timeout seconds issued at $issuetime on $issuedate
Your First OTP is $val1
```

Second message Template:

```
#set ($val2 = $otp.substring(3))
Your OTP is $otp, PAC is $pac, valid for $timeout seconds issued at $issuetime on $issuedate
Your Second OTP is $val2
```

3.2. Configure Delivery Channel

Delivery Channel needs to be appropriately configured to trigger the correct delivery handler upon OTP generation. The following are the supported delivery channels for sending OTP messages:

- **Email Deliverer**
  This delivery channel uses the SMTP (Simple Mail Transfer Protocol) server to deliver the message in an email.
- **SMPP Deliverer**
  This delivery channel makes use of SMPP (Short Message Peer-to-Peer) server via TCP/IP (Transmission Control Protocol/Internet Protocol) as a means to deliver the message in the form of SMS (Short Message Service).
- **HTTP Deliverer**
  This delivery channel uses HTTP (HyperText Transfer Protocol) server via HTTP protocol to deliver the message in an SMS (Short Message Service).

Navigate to **Administration Dashboard > Configuration > Messaging & Response** and click **Delivery Channel** on the left pane to create a new delivery channel. Next, click **Create** and select any supported delivery channels from the **Choose Delivery Channel Implementation** page. Then, click **Next** and provide the details for the delivery channel attributes before saving the changes.

All implementation types share the following attributes:

Attribute Name	Description
*Delivery Channel Id	Unique Id of delivery channel.
*Delivery Channel Name	Name of delivery channel.
Description	Description of delivery channel.
Implementation	Type of delivery channel implementation. (uneditable)

The detailed attributes differ according to the type of delivery channel selected. Below is the list of configuration attributes for each delivery channel:

Email Deliverer

Attribute	Description
Configuration Tab	
SMTP Server Information	
Host	It refers to the server name or IP Address of the SMTP server that will send the email.
Port	It refers to the port to connect to the SMTP/email server. For example, if connecting to an SMTP server over SSL, the standard port is 465. Default: 25
Secure Channel	Setting to "true" will use SMTP SSL (Secure Socket Layer)/TLS (Transport Layer Security) protocols to connect to the server. These protocols are used to secure email transmissions. Default: false
Enable STARTTLS	Set to "true" to enable STARTTLS for SMTP over SSL/TLS. STARTTLS is a way to take an existing insecure connection and upgrade it to a secure connection using SSL/TLS. However, disable this attribute if SMTPS(Simple Mail Transfer Protocol Secure) is enabled. Default: true
Connection Timeout	Socket connection timeout value in seconds. "0" value sets an infinite timeout. This is the timeout in making the initial connection, i.e. completing the TCP connection handshake. Default: 0
Read Timeout	Socket read timeout value in seconds. "0" value sets an infinite timeout. The maximum amount of time waiting to read data. Default: 0
Write Timeout	Socket write timeout value in seconds. "0" value sets an infinite timeout. The maximum amount of time waiting to write data. Default: 0
SMTP Server Authentication	
Does the server require authentication?	Setting to true will enable SMTP server authentication and will require a user name and password for authentication. Default: false

Attribute	Description
User Name	A valid email account username or email address for SMTP server authentication.
User Password	The password of the entered email account username for SMTP server authentication.
Others	
Print debug information	Setting to true will print the debug information to standard out. Default: false
Email Body Content Type	Specify the email's body content type. Either "Text" or "HTML" content. Default: Text

HTTP Deliverer

Attribute	Description
Configuration Tab	
HTTP Information	
*URL	Refer to the URL of HTTP server or gateway.
Method	Refer to the HTTP request method. Possible values: • "POST" • "GET" Default: POST
Response code for successful scenario	Specify the response code to indicate a successful scenario. Default: 200
HTTP Authentication	

Attribute	Description
Use basic authentication	Setting to "true" will enable HTTP basic access authentication. HTTP basic access authentication is a method for an HTTP user agent (e.g. a web browser) to provide a user name and password when making a request. Default: false
User Name	User name to be used for HTTP basic access authentication.
User Password	User's password to be used for HTTP basic access authentication.
HTTP Parameters	
Parameter name for recipient	Define the HTTP parameter name for the recipient of this delivery channel. **Note:** The values of this parameter are taken from the attribute values of **To**, **Cc** and **Bcc** recipients of the Message Template.
Parameter name for sender	Define the HTTP parameter name for the sender of this delivery channel. **Note:** The value of this parameter is based on the attribute value of the **From** attribute of the Message Template.
Parameter name for message body	Define the HTTP parameter name for the message body of this delivery channel. **Note:** The value of this parameter is based on the attribute value of the **Body** attribute of the Message Template.
HTTP parameter to attribute mapping	Allows setting of multiple HTTP parameters, which must be defined from Custom Attributes. The custom attributes must not be part of the pre-defined attributes used by the system. Each entry must be in this format <HTTP parameter name>=<AM's attribute name>, (e.g. "userId=attrUserId", where "userId" is the HTTP parameter name and "attrUserId" is the attribute name).

Other HTTP parameters mapping	Refer to the mapping of other HTTP parameters where each entry is a parameter name. The value of these parameters is passed by the client application in the "params" argument of AccessMatrix API. Example client code using Java:
	`Map<String, String> nParams = new HashMap<String, String>();`
	`nParams.put("phoneNumber", "123456");`
	`nParams.put("homeAddress", "ChaiChee");`
	`generateOTP(sessionToken, "", "DefaultSmsOtpPolicy", new String[]{""}, nParams);`

HTTP Proxy	
Enabled	Setting to true will enable the HTTP Proxy. Default: false
URL	Refer to the HTTP Proxy URL.

Attribute	Description
Port	Refer to the HTTP Proxy port value. **Note:** Value should not be less than 0.
User Name	Username to be used for proxy authentication.
User Password	User's password to be used for proxy authentication.
Others	
Verify Response Body	Setting to true will perform verification on the HTTP Response Body through regular expression matching. Define the regular expression in the **Response Body Regular Expression** attribute. Default: false
Response Body Regular Expression	This is used to perform regular expression matching for HTTP Response Body verification based on the Java regular expression defined, e.g. "^([A-Z0-9])." , "\b01010\b", etc. **Note:** Only required if **Verify Response Body** is set to "true".
Audit response body	Refers to the audit response body. Maximum response length is 1KB (automatically truncated), including HTML tags, newline and space characters. Default: false
Response headers to audit	Names of the response headers to be audited.
Content type character set	Refers to the character encoding to use. Default: UTF-8
TLS Version	Specifies the TLS version(s) to be used by the HttpDeliverer for HTTPS connection, e.g. TLSv1, TLSv1.1, TLSv1.2. Default: TLSv1, TLSv1.1, TLSv1.2
Connection Timeout	The maximum amount of time waiting to connect to the URL. **Note:** Specified in milliseconds. Default: 0
Response Timeout	The maximum amount of time waiting to get a response from the URL. **Note:** Specified in milliseconds. Default: 0

Notes: After entering all the mandatory information, the administrator may click on the **Simulate With Message Template** button to display "Merge HTTP Delivery Channel With Message Template Values". Next, select the message template for the simulation, and click on **Simulate** to view the simulation result.

	• To allow the use of HTTP proxy without any authentication (proxy or basic authentication), set **Use basic authentication** to "false", set the **Enabled** attribute under **HTTP Proxy** to "true", configure the **URL** and **Port** values, and leave the **User Name** and **User Password** fields to empty values.

SMPP Deliverer

Attribute Name	Description
Configuration Tab	
SMS Center	
Host	Specify the TCP/IP address or hostname of the SMPP server.
Port	Specify the TCP/IP port on the SMPP server to which the gateway should connect.
System id	Specify the user name (sometimes called System ID) for the gateway to connect to the SMPP server.
Password	Specify the password for the gateway to use when connecting to the SMPP server.
Message Parameters	
Validity Period	Expiration time in milliseconds, the message should be discarded if not delivered to the destination. Default: 180000
Delivery Status	Request delivery status. Possible values: "Not Requested", "Requested", and "Requested on failure". Default: Requested
Connection Parameters	
Session Timeout	Waiting for the delivery status, the time duration when a connection to the SMS center will remain open (in milliseconds). Default: 300000

Another delivery channel that supports sending of the SMS message is **RawHttpDeliverer**. Refer to C. Raw Http Delivery Channel for details.

3.3. Configure Event Notification Policy

To create a new event notification policy, navigate to **Administration Dashboard > Configuration > Messaging & Response** and click **Event Notification Policy** on the left pane. Next, click the **Create** button and provide the details for the event notification policy attributes before saving the changes.

Attribute Name	Description
*Policy Id	Unique Id of event notification policy.
*Event Notification Policy Name	Name of event notification policy.
Version	Event notification policy entry version.
Description	Description of event notification policy.
Account Status	The value of this attribute defines the status of the policy, whether it is activated or disabled. Any event will not trigger the disabled policy. Options: • "Activated" • "Disabled" Default: Activated
Event Filter	
Event Types	Specify the event type(s) for notification. For example, to send an SMS message upon OTP generation, click the **Add** button, set the **Event Category** to "Token" and set **Event type** to "GenerateOtp". Click **OK** to add the new event type. Refer to D. Event Types for the list of categories and event types applicable to the SMS OTP implementation.
First Template and Delivery Channel	
Message Template	Specify the message to be sent via the selected delivery channel.
Delivery Channel	Specify the delivery channel to use in sending the message. For example, the "HTTP Deliverer" channel will send the message via SMS, and the "SMTP Deliverer" channel will send the message via Email.

Note: Three different message templates and delivery channels can be configured to serve other purposes.

3.4. Configure SMS OTP Token Policy

To create a new SMS OTP Token Policy, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Click **Create** and select the "SMS OTP Token Policy" implementation from the **Choose Token Policy Implementation** page. Then, click **Next** and provide the details for the token policy attributes before saving the changes.

Attributes	Description
One-Time-Password (OTP)	
OTP length	Define the length of OTP. Range of values: 4 to 20. Default: 6
OTP character set	Define the format of OTP. • "Numeric" • "Lowercase Alphabetic" • "Uppercase Alphabetic" • "Mixedcase Alphabetic" • "Lowercase Alphanumeric" • "Uppercase Alphanumeric" • "Mixedcase Alphanumeric" Default: Numeric
Personal Assurance Code (PAC)	
PAC refers to a shorter random string to send back to the user to associate with the OTP. User will receive in their email or SMS the OTP together with the PAC. Within the application, the OTP input dialogue will also have the PAC displayed.	
PAC enabled	Define whether PAC is enabled or not. Default: true
PAC length	Define the length of PAC. Range of values: 3 to 20. Default: 4
PAC character set	Define the format of PAC. • "Numeric" • "Lowercase Alphabetic" • "Uppercase Alphabetic" • "Mixedcase Alphabetic" • "Lowercase Alphanumeric" • "Uppercase Alphanumeric"

Attributes	Description
	• "Mixedcase Alphanumeric" Default: Numeric
PAC Password Mask	PAC will be generated based on the specified password mask format. **Note:** This option will override the PAC character set. Password mask format: • A - an upper case letter • a - a lower case letter • 9 - a numeric digit • N - an alphanumeric character • - any upper case or lower case character • ! - a punctuation character • W - a upper case consonant character • w - a lower case consonant character • V - an upper case vowel character • v - a lower case vowel character Example: "AAAA9999" may generate the password STRE2435, while "AaAa9999" may generate the password ThEi7839.
Expiration	
OTP validity (in seconds)	Define the expiry of OTP in seconds. Range of values: 60 to 864000. Default: 900
Extend expiry to end of day (to 23:59:59)	If set to "true", it will set the OTP expiry to the end of the day. Default: false
Allow reuse (verify) of OTP multiple times	Define the policy to control whether the system allows or does not allow reusing OTP more than once correctly i.e. verifying OTP which is already used before. Default: false
Advanced	
Return OTP to client	Define whether OTP is exposed to the client or not besides sending OTP out via delivery channel. Default: false
Lock Token If Bad Verification Count Is More Than	Specifies the number of bad verification attempts to be exceeded before locking the token. If specified as "0", any number of bad verification attempts will not lock the token. Range of values: 0 to 30. Default: 3

Attributes	Description
Hash scheme	Hash algorithm used to store OTP hash in the database. Options: • "Salted SHA-256" • "AM4 compatible" • "SSHA-512" Default: Salted SHA-256

3.5. Configure SMS OTP Login Module

To create a new SMS OTP Login Module, navigate to **Administration Dashboard > Configuration > Authentication** and click **Login Module** on the left pane. Select the pre-created login module: "SMSOTPLogin" or click **Create** and select "iSprint SMS OTP Login" implementation from the **Choose Login Module Implementation** page. Then, click **Next** and provide the details for the login module attributes before saving the changes.

Attribute Name	Description
*Login Module Id	Unique Id of login module.
*Login Module Name	Name of login module.
Description	Description of login module.
Version	Login module entry version.
Implementation	Type of login module implementation, i.e. "i-Sprint SMS OTP Login".
Handler	Handler of the login module, i.e. "SMSOTPLogin".
Configuration Tab	
Login Configuration	
Login Policy	The login policy specifies if a bad login count should be incremented and if the login account should be locked upon exceeding the maximum bad login count, as well as other policy controls. A pre-created login policy: "DefaultBasicLoginPolicy" can be selected. Refer to *AccessMatrix Common Administration Guide - Configure Login Policy* for the attribute details of Basic Login Policy. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect.
Authentication Level	Unused. **Note:** This is an attribute of login module used for UAM product. Default: 1
Allow only one current OTP per user	If set to "true", then the user will only have one current OTP and each newly generated OTP will override it. Default: false
Check user's 'Email' or 'Mobile' attribute	Check whether the user's email or mobile attribute is empty or not. Options: • "-" - No checking is done. • "Email" - Check whether the user's email attribute is null. If null, throws "NoEmailForOTPDeliveryException" exception.

Attribute Name	Description
	• "Mobile" - Check whether the user's 'mobile' attribute is null. If null, throws "NoMobileForOTPDeliveryException" exception. **Note:** This option will not check for the custom attributes or email/mobile provided from API parameters.
Maximum OTPs Within Duration	
Maximum OTPs	This setting specifies the maximum number of OTPs allowed to be generated within a specific duration. The duration is configured in the **Duration (seconds)** attribute below. Setting the value of **Maximum OTPs** to "0" means there is no limit to the number of OTPs that can be generated within any given duration. Default: 0
Duration (seconds)	If the number of OTPs generated within this duration is exceeded, any further attempts will fail and an error will be thrown. Once this happens, further OTP attempts will only be allowed after this much time has elapsed since the first successfully generated OTP. For example, setting **Maximum OTPs** to "10" and **Duration (seconds)** to "3600" means that the user cannot generate more than 10 OTPs within an hour. If there is an 11th attempt during this period, it will fail and an error will be thrown. The user must then wait an hour after the 1st successful attempt before they can generate another OTP. Default: 3600
Token Policy	
SMS OTP Token Policy	The SMS OTP token policy to use for this login module, e.g. "DefaultSmsOtpPolicy".
Default Event Notification Policy	
Event Notification Policy	It is an event-based trigger for sending email or SMS OTP message depending on the delivery channel configured for this notification policy. For example, sending OTP to the login users when Generate OTP event is triggered. **Note:** This is only the policy that will be invoked when resetting the password.

3.6. Configure Authentication Realm

To create a new Authentication Realm, navigate to **Administration Dashboard > Configuration > Authentication** and click **Authentication Realm** on the left pane. Next, click **Create** and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm.
*Authentication Realm Name	Name of authentication realm.
Description	Description of authentication realm.
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in user store(s) or other validation methods).
First(1st) Login Module	The first login method to use for user authentication.
Second(2nd) Login Module	Optional subsequent user authentication method.
Third(3rd) Login Module	Optional subsequent user authentication method required.
Fourth(4th) Login Module	Optional subsequent user authentication method required.
Fifth(5th) Login Module	Optional subsequent user authentication method required.

An authentication realm can be configured to provide a chained authentication known as multiple-factor authentication (MFA) of up to five authentication factors. For example, a 2-step MFA can be configured where the username and static password are sent for authentication first before prompting the user to enter the i-Sprint SMS OTP. First, it can be set by setting the "First(1st) Login Module" to use "System Password Basic" or "Ldap Login". Then, set the "Second(2nd) Login Module" to use the **i-Sprint SMS OTP Login** login module, e.g. "SMSOTPLogin".

3.7. Configure User Accounts

A user may belong to a default store or external user store. Refer to *AccessMatrix Common Administration Guide - Manager User Stores* to know about the different types of user stores and how to configure them.

To create a new user in the default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Then, provide values to the following attributes necessary for SMS OTP implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Attributes Tab	
This tab contains system pre-defined attributes: "Email" and "Mobile". Ensure that the "Email" value is provided when configuring to send an OTP message via Email and provide the "Mobile" value when configuring to send an OTP message via SMS.	
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to the user. First, click **Add** to display the **Choose Authentication Realm** dialog box. Next, select the authentication realm created in the previous section and click **OK**. Both the authentication realm and the login module(s) associated with it will be assigned to the user.	

The **Reset OTP History** button is also provided on the **View User** page. Using this button clears the history of recently generated OTPs.

This can be useful when a user has exceeded the maximum number of OTPs allowed to be generated within a given duration. If the user cannot wait for the restriction to expire, an admin can manually reset the OTP history to allow immediate further OTP generation attempts.

> **Note:** The maximum number of OTPs that can be generated within a given duration is configured in the SMS OTP login module settings. Details on the relevant attributes (**Maximum OTPs** and **Duration**) can be found in the Configure SMS OTP Login Module page.

To use this feature:

1. Click the **Reset OTP History** button.

   > **Note:** To use this feature, a valid authentication realm with the SMS OTP login module must be assigned to the user. If not, a dialog box will be displayed to inform the admin that no such login module can be found for the user.

2. Select the SMS OTP login module for which the OTP history should be reset.
3. Click **Reset**. A confirmation message is displayed if the reset is successful.

Refer to *AccessMatrix Common Administration Guide - Manager Users* for more information about users in general.

3.8. Configure Cache Replication

By default, SMS OTP uses MemoryTokenStore, which does not persist in the database. If the project has session stickiness that can ensure that generateOtp and verifyOtp are served by the same AccessMatrix server, replication is not required. This configuration is also more efficient.

However, if there is no session stickiness without replication, the verifyOtp may be served by another AccessMatrix server instance that does not have the cache entry to be verified. This causes an issue such as tokens not being found. To allow generateOtp and verifyOtp to be done by different AccessMatrix server instances, MemoryTokenStore has been configured for replication. See below.

MemoryTokenStore in am5-ehcache.xml

XML

```
<!-- SMS OTP cache -->

<!-- For sms otp token in UAS SMS OTP, set TTI to 0, set TTL
to be slightly longer than otp expiry policy,
     IMPORTANT: replicatePuts and replicateUpdatesViaCopy
must be set to true for SMS OTP Token if web-tier session
stickiness is not present  -->
<cache name="MemoryTokenStore"
    maxElementsInMemory="30000"
    eternal="false"
    timeToIdleSeconds="0"
    timeToLiveSeconds="900"
    overflowToDisk="false">
    <!-- if generateOtp/verifyOtp could end up with different
instances of AM5 servers, replication must be enabled and
tested -->
    <cacheEventListenerFactory
class="net.sf.ehcache.distribution.RMICacheReplicatorFactory"
       properties="replicateAsynchronously=true,
                   replicatePuts=true,
                   replicateUpdates=true,
                   replicateUpdatesViaCopy=true,
                   replicateRemovals=true,

asynchronousReplicationIntervalMillis=100"/>
     </cache>
```

If SystemTokenStore is used instead of MemoryTokenStore, then the cache is not used and there is no need to configure replication for it.

4. UAS SMS OTP Implementation Using API

SMS OTP API Use Cases

The primary use cases of UAS SMS OTP are:

- Authentication (typically as a 2nd-factor authentication mechanism in an internet application).
- Transaction authorization (typically to authorize set up of sensitive operation such as fund transfer target account in internet banking).
- SMS OTP Invalidation

The UAS SMS OTP solution is normally used as a second-factor authentication (2FA) although it is also possible to use it as a standalone (e.g. for OTP verification only). In these use cases, an OTP will be generated and sent via SMS or email, depending on which delivery method is in place.

There are 2 ways to do SMS OTP verification. First is to use /authn/login with an authentication realm that is configured with 2FA and the second one is to use /token/generateOTP followed by /token/verifyOTP. When 2FA is setup (i.e. second login module is i-Sprint SMS OTP Login), calling the login API for 1FA (i.e. /authn/login) will trigger SMS-OTP generation and delivery. But when there is only i-Sprint SMS OTP Login module configured in the authentication realm, call the same API and pass in an empty password to trigger SMS OTP generation and delivery.

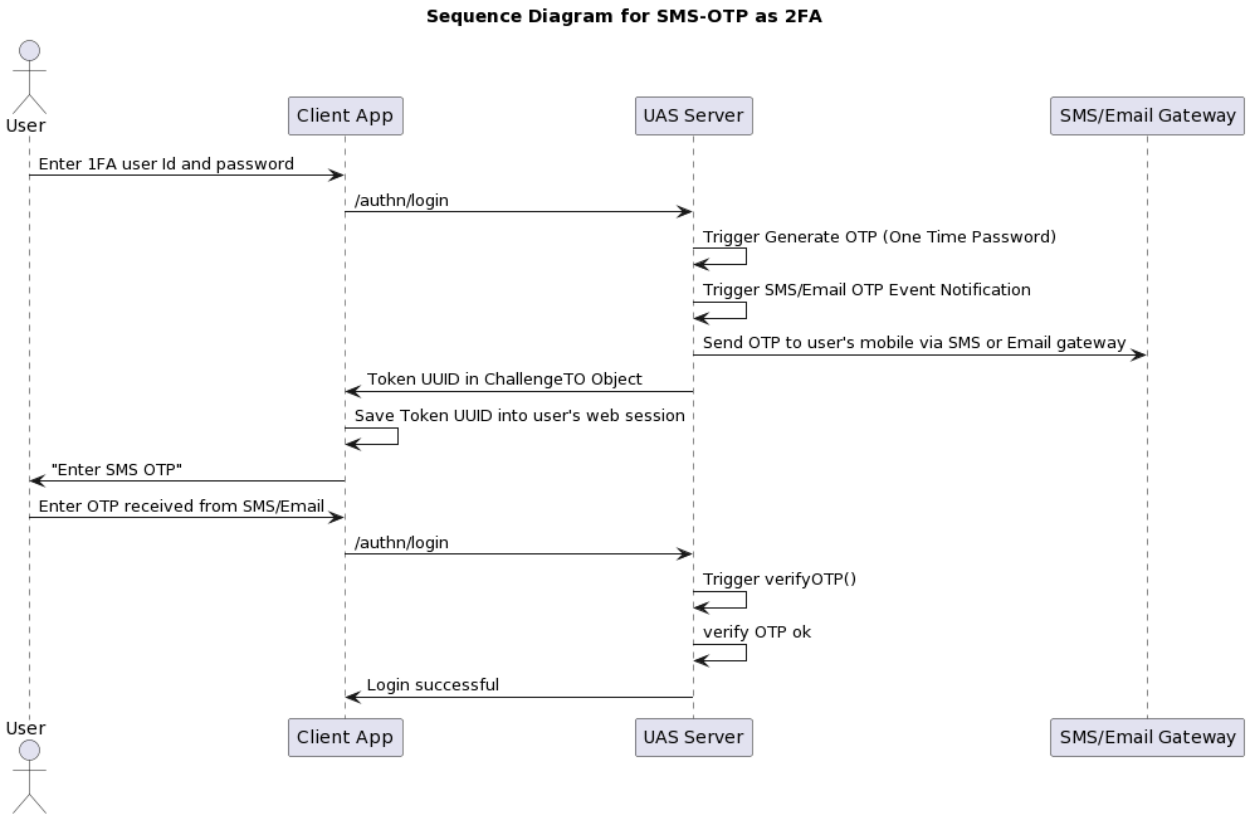
The following sections describe the APIs that can be used for each use case scenario.

4.1. Two-Factor Authentication

Two-factor authentication (also known as 2FA) is a type, or subset, of multi-factor authentication. It is a method of confirming users' claimed identities by combining two different factors: 1) something they know, 2) something they have, or 3) something they are.

Sequence Diagram for SMS OTP 2FA Login

- During user login, authentication realm is one of the parameters passed in invoking the /authn/login REST API.
- If a password is left blank and the authentication realm is set properly, SMS OTP generation will be triggered automatically for the 2nd Factor Login.
- Then, /authn/login is invoked again, this time with the SMS OTP received from SMS/Email as the password and the challenge is the challenge object received initially from the 1st login call.
- SMS OTP authentication realm is again passed in as one of the parameters.



REST Sample Code

There are few differences between the payload of the first call and the second (and any subsequent) calls when invoking /authn/login.

- Use /authn/login to perform SMS OTP two-factor authentication.
- Refer to the *AccessMatrix REST API Specification* for more details.

2FA Login API Calls

First Login Call

- "PwSms" is a sample realm with 2FA of system password basic + SMS OTP login.
- Password argument can be blank ("") if the realm specified has the SMS OTP login module only.
- Password argument is mandatory if the SystemPasswordBasic login module is used with SMS OTP login module for the 2FA realm.
- 1st login will trigger a generateChallenge() at server side to generate ChallengeTO object and will be required for the 2nd login.

URL: /authn/login

Request

JSON

```
{
    "sessionToken": "",
    "userId": "pwsmsuser",
    "realmId": "PwSms",
    "password": "password"
}
```

Response

JSON


```
{
"result": {
    "authenticated": false,
    "sessionIID": "TkLCzCvF-Eo4EdFNs91ZjQ",
    "lastAccessTime": 1560390600419,
    "realmId": "PwSms",
    "sessionFlags": 0,
    "serverId": "ChinKai-PC",
    "userId": "pwsmsuser",
    "realm.lastLoginSuccessTime": 1559292719000,
    "loginTime": 1560390599866,
    "sessionToken":
"10001aAuIqi3Qs2bfTjJh4aeNvcHLjh15hYfzbdBasXTSCQG2NW2oi6f2z5JiZybDrU_teITSAv
```

2FA Login API Calls


```
PVie44mQGGf9IzVChG3TFIO4nnuvGYZJ4rAN0kAQNZThM4IhYlc4dt5PeqbNOiHtWuyTzpq2wqrq
nyTkQ1eVEPqkV8dKAtpRH6PYOqDBMM4S-
R8tMaj9kAcyPJalBQa2gaU8UZennt2PIciI44ljB5Vp77Ja4-LfDdqBXcryxWFf-15qzObg2wM-
JoK10GVIEtB4iR5zab8VpI0JM2AKKXy6FXaz3VV_cdWAEh2tP7x41sKW8rTO-arQjxU",
    "challenge": {
        "challengeToken":
"0001GmfvpRHgGmxqTacE9N82nrlYIGPf25FkNu34MToZiWGik4I4hbD1GgXS1kvlYXEfvvCkw70
Q",
        "authenticationCode": 18888,
        "message": "PAC=4511",
        "params": {
            "expireddate": "Jun 13, 2019",
            "pac": "4511",
            "returnOTPToClient": true,
            "issuetime": "9:50:00 AM",
            "expiredtime": "10:05:00 AM",
            "expiredTimeMilis": "1560391500390",
            "otp": "21383370",
            "issueTimeMilis": "1560390600390",
            "deliverySearchKey": "MemoryTokenStore:Q2LngxaUqB",
            "tokenUUID": "MemoryTokenStore:Q2LngxaUqB",
            "issuedate": "Jun 13, 2019",
            "timeout": "900"
        }
    },
    "responses": {
    },
    "userUUID": "l4gbagIa0p",
    "loginModuleStates": [
        {
            "authenticated": true,
            "canChangePassword": true,
            "id": "SystemPasswordBasic",
            "forceChangePassword": true,
            "forceChangePasswordReason": 3
        },
        {
            "authenticated": false,
            "canChangePassword": false,
            "id": "SMSOTPLogin"
        }
    ],
    "attr.regenerated": "1560390600428"
    },
    "amProcessingTimeMillis": 655
}
```


Second Login Call

- SMS OTP is passed in together with Challenge token from the 1st login to perform the 2nd login.

URL: /authn/login

2FA Login API Calls

Request

JSON


```
{
    "sessionToken":
"10001aAuIqi3Qs2bfTjJh4aeNvcHLjh15hYfzbdBasXTSCQG2NW2oi6f2z5JiZybDrU_teITSAv
PVie44mQGGf9IzVChG3TFIO4nnuvGYZJ4rAN0kAQNZThM4IhYlc4dt5PeqbNOiHtWuyTzpq2wqrq
nyTkQ1eVEPqkV8dKAtpRH6PYOqDBMM4S-
R8tMaj9kAcyPJalBQa2gaU8UZennt2PIciI44ljB5Vp77Ja4-LfDdqBXcryxWFf-15qzObg2wM-
JoK10GVIEtB4iR5zab8VpI0JM2AKKXy6FXaz3VV_cdWAEh2tP7x41sKW8rTO-arQjxU",
    "userId": "pwsmsuser",
    "realmId": "PwSms",
    "password": "21383370",
    "challengeToken":
"0001GmfvpRHgGmxqTacE9N82nrlYIGPf25FkNu34MToZiWGik4I4hbD1GgXS1kvlYXEfvvCkw70
Q"
}
```


Response

JSON


```
{
    "result": {
        "authenticated": true,
        "sessionIID": "TkLCzCvF-Eo4EdFNs91ZjQ",
        "lastAccessTime": 1560390600604,
        "realmId": "PwSms",
        "sessionFlags": 1,
        "loginUserAgent": "",
        "serverId": "ChinKai-PC",
        "userId": "pwsmsuser",
        "realm.lastLoginSuccessTime": 1559292719000,
        "loginTime": 1560390599866,
        "sessionToken": "100015YZWeYN3DYEHrTnP6JvOCLMplF11x6PNcjZz23fa8gr-
36Urqj1wRh2glzZkJLZCCAo4HbwtXvf13JcrZNVWL2U7m4R2Hc4WmFa7rO1lsJD61220w-
kbD8dE5I5KzDdeCaFVG4Rx-3iIX7Gg5Wfb-
JKgX9vwH7ZW50xHbddeRHstxfe4qooG6qNHpz5o0Wsb0Y2K7YDWlw6QMiSEg1-
Wz0u4ywKx1NUUed3K2YLg1ZCiTJKEvKdo7gNV8M1Wb-
vpQ54hBuLSSDNUH5MjO7C14TMNXN_k5yQWBXzQNq3uuPF8RqhDMryJpW1zSjGNMYw_7a5QfKNX9e
c",
        "responses": {
        },
    "userUUID": "l4gbagIa0p",
    "loginLocation": "",
    "loginModuleStates": [
        {
            "authenticated": true,
            "canChangePassword": true,
            "id": "SystemPasswordBasic",
            "forceChangePassword": true,
            "forceChangePasswordReason": 3
        },
        {
            "authenticated": true,
```

2FA Login API Calls

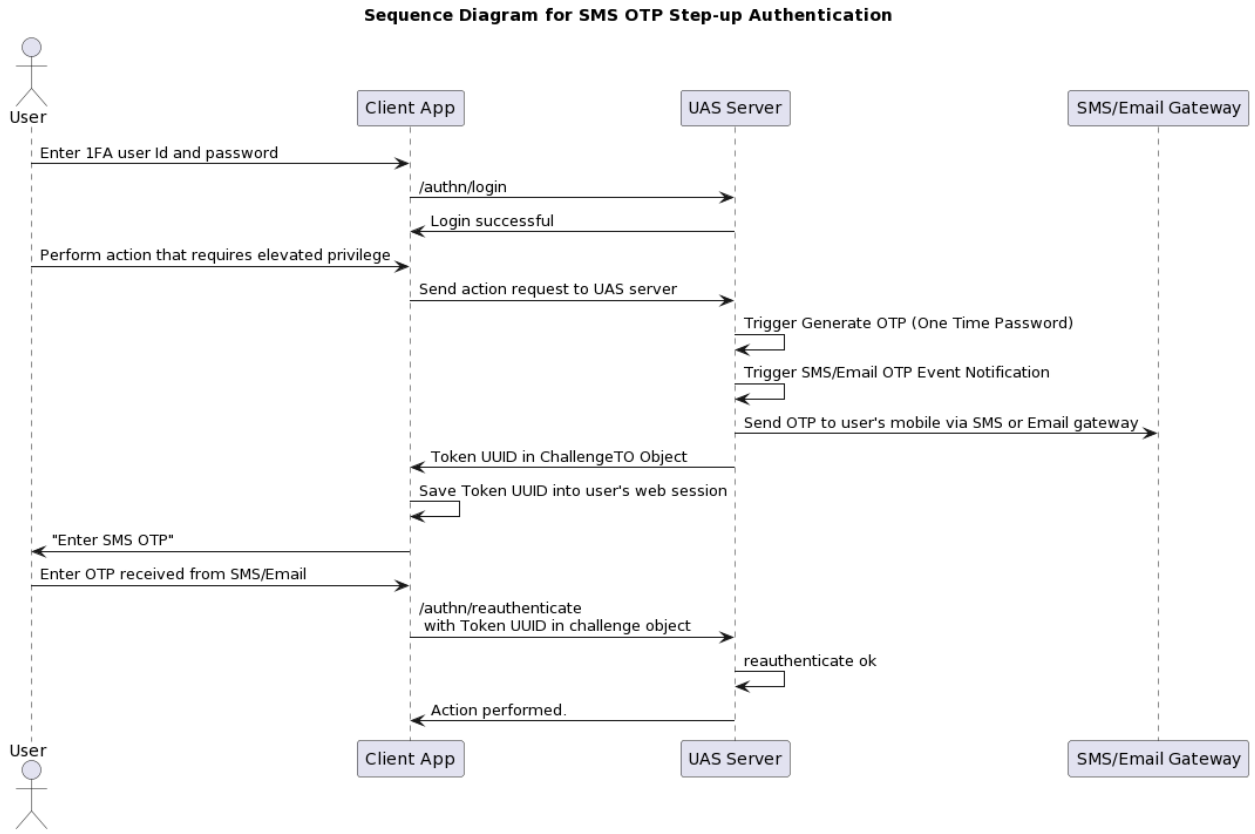

```
            "canChangePassword": false,
            "id": "SMSOTPLogin"
        }
    ],
    "attr.regenerated": "1560390600655",
    "remoteAddr": ""
    },
    "amProcessingTimeMillis": 89
}
```

4.2. Step-up Authentication

Step-up authentication is the process by which a user is challenged to produce additional forms of authentication to provide a higher level of assurance that he is who he claims to be. For example, a user asking to transfer a large sum of money may be challenged with step-up authentication to produce another one-time passcode to complete the request.

Step-up authentication can include any number of authentication methods, including MFA, one-time code over SMS, knowledge-based authentication (KBA), biometrics, etc.

Sequence Diagram for SMS OTP Step-up Authentication



REST Sample Code

Two separate methods are invoked. The first is /token/generateOTP for generating the SMS OTP, and the second is /authn/reauthenticate which reauthenticates the caller.

- Use /token/generateOTP to generate an SMS OTP token.

- Use /authn/reauthenticate to reauthenticate existing sessions to a higher level login module. This is normally done when the user needs to be authenticated with a more secure module before the user can access some higher privileged functions.
- Refer to the *AccessMatrix REST API Specification* for more details.

Step-up Authentication API Calls

Generate OTP Call

- "SMSOtp" is a sample event notification policy via SMS.

URL: /token/generateOTP

Request

JSON


```
{
  "sessionToken": "100015YZWeYN3DYEHrTnP6JvOCLMplF11x6PNcjZz23fa8gr-
36Urqj1wRh2glzZkJLZCCAo4HbwtXvf13JcrZNVWL2U7m4R2Hc4WmFa7rO1lsJD61220w-
kbD8dE5I5KzDdeCaFVG4Rx-3iIX7Gg5Wf_b-
J_KgX9vwH7ZW50xHbddeRHstxfe4qooG6qNHpz5o0Wsb0Y2K7YDWlw6QMiS_Eg1-
Wz0u4ywKx1NUUed3K2YLg1ZCi_TJKEvKdo7gNV8M1Wb-
vpQ54hBuLSSDNUH5MjO7C14TMNXN_k5yQWBXzQNq3uuPF8RqhDMryJpW1zSjGNMYw_7a5QfKNX9e
c",
  "policyId": "",
  "params": {
    "eventNotificationId": "SMSOtp"
  }
}
```


Response

JSON


```
{
    "result": {
        "challengeToken":
"0001WxM_kAeklaCuEI8CsVnIRAQxupf8Pj3OtBT345NP12BK4KSp4V7hVOXbqnEBtjzYXmJSUQ
-eaw",
        "authenticationCode": -1,
        "message": "PAC=8428",
        "params": {
            "expireddate": "Jun 13, 2019",
            "pac": "8428",
            "returnOTPToClient": true,
            "issuetime": "9:50:00 AM",
            "expiredtime": "10:05:00 AM",
            "expiredTimeMilis": "1560391500717",
            "otp": "91999184",
            "issueTimeMilis": "1560390600717",
            "deliverySearchKey": "MemoryTokenStore:vnn_34aUqE",
            "tokenUUID": "MemoryTokenStore:vnn_34aUqE",
```

Step-up Authentication API Calls


```
            "issuedate": "Jun 13, 2019",
            "timeout": "900"
        }
    },
    "amProcessingTimeMillis": 8
}
```


Reauthenticate Call

- SMS OTP is passed in together with the challenge token from the Generate OTP call to perform reauthentication.

URL: /authn/reauthenticate

Request

JSON


```
{
  "sessionToken": "authenticated session token",
  "loginModuleId": "SMSOTPLogin",
  "password": "91999184",
  "challengeToken":
"0001WxM_kAeklaCuEI8CsVnIRAQxupf8Pj3OtBT345NP12BK4KSp4V7hVOXbqnEBtjzYXmJSUQ
-eaw"
}
```


Response

JSON


```
{
    "result": {
        "authenticated": true,
        "sessionIID": "TkLCzCvF-Eo4EdFNs91ZjQ",
        "lastAccessTime": 1560390600816,
        "realmId": "PwSms",
        "sessionFlags": 1,
        "loginUserAgent": "",
        "serverId": "ChinKai-PC",
        "userId": "pwsmsuser",
        "realm.lastLoginSuccessTime": 1559292719000,
        "loginTime": 1560390599866,
        "sessionToken": "10001hUX9lxY5epvsJvjA8lNoMwCE9UMRmJtSKb-
nIEpnfnHGej-0HT3bGXTDFuY1nVzZls99pKrUsdRE2jLXLrys7-i7LJC3zvsW-
H0_wTCP1BQcxBIByqO5RFiSfYTtabsXPvmAlxuVttZhcQJoIsHyjFcQheNIpP9d9AO6gr9ql_LxZ
09kWm2TuZRbp7V2rZ-
RjkLjSFXn07wG8CBGiZ6229mkwx8_Rl_sSZwjfl8LPt3VcMZcS2kOwC6ZOTdrm0WmkZMSqNflJPL
vhwjl7VP3u0NFpbryBNUQ65hx5iW8yCX0cMz15qXvMUNdj_NkrKTe-
22RG7vDsk1yGpgYIkTbyaJmpe8KKet0LtRMKes",
        "responses": {
        },
```

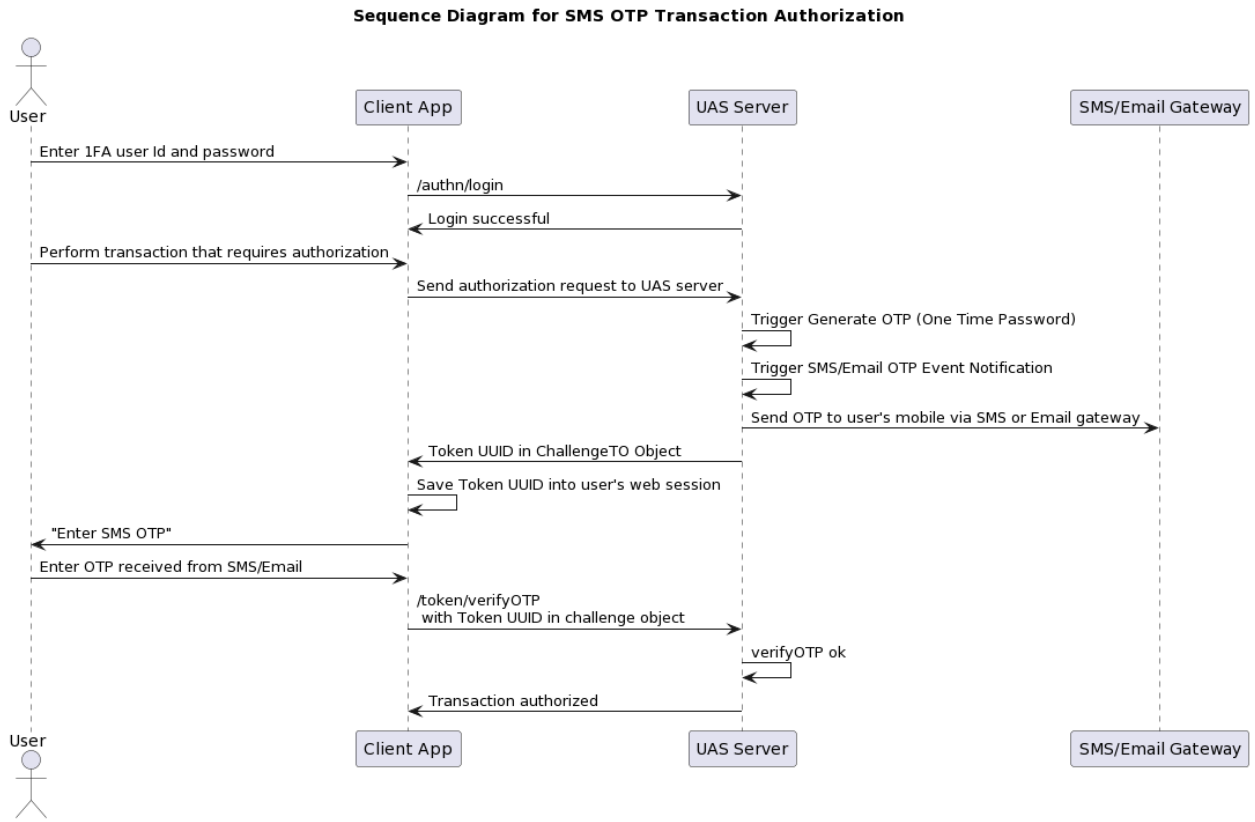
Step-up Authentication API Calls


```
        "userUUID": "l4gbagIa0p",
        "loginLocation": "",
        "loginModuleStates": [
            {
                "authenticated": true,
                "canChangePassword": true,
                "id": "SystemPasswordBasic",
                "forceChangePassword": true,
                "forceChangePasswordReason": 3
            },
            {
                "authenticated": true,
                "canChangePassword": false,
                "id": "SMSOTPLogin"
            },
            {
                "authenticated": true,
                "canChangePassword": false,
                "id": "ongSms"
            }
        ],
        "attr.regenerated": "1560390600817",
        "remoteAddr": ""
    },
    "amProcessingTimeMillis": 30
}
```

4.3. Transaction Authorization

Transaction authorization is the process by which a user is challenged with a one-time passcode to authorize the transaction and ensure it has been requested and verified by the user.

Sequence Diagram for SMS OTP Transaction Authorization



REST Sample Code

Transaction authorization is a two-step process. The first is to invoke /token/generateOTP, and the second is to invoke /token/verifyOTP.

- Use token/generateOTP to generate an SMS OTP token.
- Typically, an event notification policy, a message template and a delivery channel will be configured to handle the Generate OTP event to send out the OTP to the user.
- Use token/verifyOTP to verify the OTP.
- Refer to the *AccessMatrix REST API Specification* for more details.

Transaction Authorization API Calls

Generate OTP Call

- "SMSOtp" is a sample event notification policy via SMS.

URL: /token/generateOTP

Request

JSON


```
{
  "sessionToken": "10001hUX9lxY5epvsJvjA8lNoMwCE9UMRmJtSKb-nIEpnfnHGej-
0HT3bGXTDFuY1nVzZls99pKrUsdRE2jLXLrys7-i7LJC3zvsW-
H0_wTCP1BQcxBIByqO5RFiSfYTtabsXPvmAlxuVttZhcQJoIsHyjFcQheNIpP9d9AO6gr9ql_LxZ
09kWm2TuZRbp7V2rZ-
RjkLjSFXn07wG8CBGiZ6229mkwx8_Rl_sSZwjfl8LPt3VcMZcS2kOwC6ZOTdrm0WmkZMSqNflJPL
vhwjl7VP3u0NFpbryBNUQ65hx5iW8yCX0cMz15qXvMUNdj_NkrKTe-
22RG7vDsk1yGpgYIkTbyaJmpe8KKet0LtRMKes",
  "policyId": "",
  "params": {
    "eventNotificationId": "SMSOtp"
  }
}
```


Response

JSON


```
{
    "result": {
        "challengeToken":
"0001WgtbJrk68oISf9XUA5W563uY3cAg39r-
NdQ7hFEj4cOp60Q4SFChcq4TyJQXFMhtUENFTfJUbg",
        "authenticationCode": -1,
        "message": "PAC=8428",
        "params": {
            "expireddate": "Jun 13, 2019",
            "pac": "8428",
            "returnOTPToClient": true,
            "issuetime": "9:50:00 AM",
            "expiredtime": "10:05:00 AM",
            "expiredTimeMilis": "1560391500717",
            "otp": "91999184",
            "issueTimeMilis": "1560390600717",
            "deliverySearchKey":
"MemoryTokenStore:vnn_34aUqE",
            "tokenUUID": "MemoryTokenStore:vnn_34aUqE",
            "issuedate": "Jun 13, 2019",
            "timeout": "900"
        }
    },
    "amProcessingTimeMillis": 8
}
```

Transaction Authorization API Calls

Verify OTP Call

- SMS OTP is passed in together with token UUID from the Generate OTP call to perform authorization.

URL: /token/verifyOTP

Request

JSON


```
{
  "sessionToken": "10001hUX9lxY5epvsJvjA8lNoMwCE9UMRmJtSKb-nIEpnfnHGej-
0HT3bGXTDFuY1nVzZls99pKrUsdRE2jLXLrys7-i7LJC3zvsW-
H0_wTCP1BQcxBIByqO5RFiSfYTtabsXPvmAlxuVttZhcQJoIsHyjFcQheNIpP9d9AO6gr9ql_LxZ
09kWm2TuZRbp7V2rZ-
RjkLjSFXn07wG8CBGiZ6229mkwx8_Rl_sSZwjfl8LPt3VcMZcS2kOwC6ZOTdrm0WmkZMSqNflJPL
vhwjl7VP3u0NFpbryBNUQ65hx5iW8yCX0cMz15qXvMUNdj_NkrKTe-
22RG7vDsk1yGpgYIkTbyaJmpe8KKet0LtRMKes",
  "UUID": "MemoryTokenStore:vnn_34aUqE",
  "OTP": "91999184"
}
```


Response

JSON


```
{
    "result": {
        "authenticationCode": -1,
        "params": {
            "tokenSerNum": "_WQyzUaUqE",
            "tokenStatus": "Activated",
            "vendorId": "SMS",
            "otp": "91095525",
            "tokenUUID": "MemoryTokenStore:_WQyzUaUqE",
            "tokenModel": "Default"
        }
    },
    "amProcessingTimeMillis": 9
}
```


/password/verify

- Use /password/verify to verify a password against a given login module.
- For this case, the login account will be disabled/locked if the wrong OTP input reaches the maximum bad login count.
- Refer to the *AccessMatrix REST API Specification* for more details.

Transaction Authorization API Calls

URL: /password/verify

Request

JSON


```
{
    "sessionToken":
"100015sNcGPHw8MGz8O8uTxt5LII58HdEMSjqAZlfGHEONVIPomWDrMSdnfLsvR9t2jQ9nSgSCI
M2h6mIDUjz4MxMuwz5eryCUNDywIDGnQ2kK2os57cv_11ONghlt5TJ6YG7Z2a3vjAJLASx-
M0MKRnNeZolIw-wzDqIujMZIQAUXv9kX01itW5AH-ANfcrdwNhsmDg32Ekl4YPleI4HwUK0--
OoZtss3fyKJmmfhDXvKS3k8-
r_0I8aae0mBO_WElBd1Kyj_3kEgql4t6T3yVxHz95mtKVPaC6c9FFZFuXy5FrdB5UyQ20FH009",
    "userId": "pwsmsuser",
    "password": "69833502",
    "loginModuleId": "testSmsLoginMod",
    "challengeToken": "0001Aqa5VZYtWCa3vjCghhtl_WGTBp_cDsKQA0_MY8WLE6ozd-
qb_7jrc8dLdhtc8Lk_8CmJpBzDBw"
}
```


Response

JSON


```
{
    "result": {
        "responses": {
        },
        "loginModuleStates": [
            {
                "authenticated": true,
                "canChangePassword": false,
                "id": "testSmsLoginMod",
                "loginModuleHandlerClass": "SMSOTPLogin"
            }
        ]
    },
    "amProcessingTimeMillis": 7
}
```

4.4. SMS OTP Invalidation

SMS OTP will automatically expire after a preconfigured validity period set in i-Sprint SMS OTP Token policy at **Administration Dashboard > Security Policy > Token > Token Policy > Attributes tab > Expiration > OTP Validity (in seconds)**. However, you can explicitly invalidate an SMS OTP by calling TokenService/deleteToken.

REST Sample Code

SMS OTP Invalidation Sample Request (REST)

Request

JSON


```
{
    "sessionToken":
"100015sNcGPHw8MGz8O8uTxt5LII58HdEMSjqAZlfGHEONVIPomWDrMSdnfLsvR9t2jQ9nSgSCI
M2h6mIDUjz4MxMuwz5eryCUNDywIDGnQ2kK2os57cv_11ONghlt5TJ6YG7Z2a3vjAJLASx-
M0MKRnNeZolIw-wzDqIujMZIQAUXv9kX01itW5AH-ANfcrdwNhsmDg32Ekl4YPleI4HwUK0--
OoZtss3fyKJmmfhDXvKS3k8-
r_0I8aae0mBO_WElBd1Kyj_3kEgql4t6T3yVxHz95mtKVPaC6c9FFZFuXy5FrdB5UyQ20FH009",
    "UUID": "MemoryTokenStore:7CmVFGbs3v"
}
```


Response

JSON


```
{
    "result": {
        "responses": [
            {
            }
        ]
    },
    "amProcessingTimeMillis": 171
}
```

4.5. Additional Verification Information

For extra security, it is possible to associate the SMS OTP request with certain user session/device information, which in the end will also be verified as part of the OTP verification. For example, as part of the OTP request generation API call, the server-side application can provide the registered device ID. Then, UAS will need the same device ID to be provided as part of the OTP verification where the device ID for the verification is acquired by the application's front end. The OTP verification will only be successful if the device ID provided during verification matches the device ID provided during generation. This device ID can also be substituted with other information such as the hash of the user's web session ID.

- API is the same as shown in previous sections using the Token Service or Authentication Service. To use this additional verification information, the parameter addlVerificationId must be provided during generation and during verification.
- Use token/generateOTP to generate an SMS OTP token.
- Use token/verifyOTP to verify the OTP.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Additional Verification API Calls

Generate OTP Call

URL: /token/generateOTP

Request

JSON


```
{
    "sessionToken": "10001hUX9lxY5epvsJvjA8lNoMwCE9UMRmJtSKb-nIEpnfnHGej-
0HT3bGXTDFuY1nVzZls99pKrUsdRE2jLXLrys7-i7LJC3zvsW-
H0_wTCP1BQcxBIByqO5RFiSfYTtabsXPvmAlxuVttZhcQJoIsHyjFcQheNIpP9d9AO6gr9ql_LxZ
09kWm2TuZRbp7V2rZ-
RjkLjSFXn07wG8CBGiZ6229mkwx8_Rl_sSZwjfl8LPt3VcMZcS2kOwC6ZOTdrm0WmkZMSqNflJPL
vhwjl7VP3u0NFpbryBNUQ65hx5iW8yCX0cMz15qXvMUNdj_NkrKTe-
22RG7vDsk1yGpgYIkTbyaJmpe8KKet0LtRMKes",
    "policyId": "",
    "params": {
        "addlVerificationId": "some session id"
    }
}
```

Additional Verification API Calls

Response

JSON


```
{
    "result": {
        "challengeToken":
"0001WgtbJrk68oISf9XUA5W563uY3cAg39r-
NdQ7hFEj4cOp60Q4SFChcq4TyJQXFMhtUENFTfJUbg",
        "authenticationCode": -1,
        "message": "PAC=9404",
        "params": {
            "expireddate": "Jun 14, 2019",
            "pac": "9404",
            "returnOTPToClient": true,
            "issuetime": "11:31:39 AM",
            "expiredtime": "11:46:39 AM",
            "expiredTimeMilis": "1560483999640",
            "otp": "85153338",
            "issueTimeMilis": "1560483099640",
            "deliverySearchKey":
"MemoryTokenStore:7CmVFGbs3v",
            "tokenUUID": "MemoryTokenStore:7CmVFGbs3v",
            "issuedate": "Jun 14, 2019",
            "timeout": "900"
        }
    },
    "amProcessingTimeMillis": 4
}
```


Verify OTP Call

URL: /token/verifyOTP

Request

JSON


```
{
    "sessionToken": "10001hUX9lxY5epvsJvjA8lNoMwCE9UMRmJtSKb-nIEpnfnHGej-
0HT3bGXTDFuY1nVzZls99pKrUsdRE2jLXLrys7-i7LJC3zvsW-
H0_wTCP1BQcxBIByqO5RFiSfYTtabsXPvmAlxuVttZhcQJoIsHyjFcQheNIpP9d9AO6gr9ql_LxZ
09kWm2TuZRbp7V2rZ-
RjkLjSFXn07wG8CBGiZ6229mkwx8_Rl_sSZwjfl8LPt3VcMZcS2kOwC6ZOTdrm0WmkZMSqNflJPL
vhwjl7VP3u0NFpbryBNUQ65hx5iW8yCX0cMz15qXvMUNdj_NkrKTe-
22RG7vDsk1yGpgYIkTbyaJmpe8KKet0LtRMKes",
    "UUID": "MemoryTokenStore:7CmVFGbs3v",
    "OTP": "85153338",
    "params": {
        "addlVerificationId": "some session id"
    }
}
```


Response

Additional Verification API Calls

JSON


```
{
    "result": {
        "authenticationCode": -1,
        "params": {
            "tokenSerNum": "7CmVFGbs3v",
            "tokenStatus": "Activated",
            "vendorId": "SMS",
            "otp": "85153338",
            "tokenUUID": "MemoryTokenStore:7CmVFGbs3v",
            "tokenModel": "Default"
        }
    },
    "amProcessingTimeMillis": 3
}
```

5. UAS SMS OTP Housekeeping Policy

The housekeeping policy contains the rules on how to housekeep the SMS token data in the database. To view the housekeeping policy, navigate to **Administration Dashboard > System > Housekeeping** and click H**ousekeeping Policy** on the left pane. Select the **UAS Tab** and click the **Edit** button. Save the changes after modification.

Below is the list of SMS token-related housekeeping attributes:

Attribute Name	Description
UAS Tab	
Tokens	
Purge SMS tokens	Determine if the SMS tokens will be purged or not. Default: true
Purge SMS tokens older than (days)	Only those SMS tokens older than the specified value will be purged. Value should be from 1 to 365. Default: 3

6. UAS SMS OTP Token Reports

The following reports can be generated to provide SMS OTP token-related activities and information.

System & Admin

- **Monitoring Report**
  This report provides information regarding the delivery channel. To view this report, navigate to **Report Dashboard > System & Admin > Real Time** and click **Monitoring** on the left pane.
- **Events Report**
  This report provides information regarding the events incurred in the AccessMatrix system within the specified range of date and time. Event categories can be selected to generate specific event activities such as generation and verification of OTP. To view this report, navigate to **Report Dashboard > System & Admin > Historical** and click **Events** on the left pane. Input the report criteria, select the report format (i.e. HTML, PDF or CSV) and click the **Generate** button.

Segment & User

- **User Activities Report**
  This report provides information about the user activities based on certain event categories, e.g. Authentication, Token. To view this report, navigate to **Report Dashboard > Segment & User > Historical** and click **User Activities** on the left pane. Input the report criteria, select the report format (i.e. HTML, PDF or CSV) and click the **Generate** button. Input the report criteria, select the report format (i.e. HTML, PDF or CSV) and click the **Generate** button.

Authentication

- **Login Violation Report**
  This report provides information about the login violation incurred by the user based on certain violation types, e.g. Denied by OTP reuse is not allowed policy, OTP/signature/challenge response/password verification failure. To view this report, navigate to **Report Dashboard > Authentication > Historical** and click **Login Violation** on the left pane. Input the report criteria, select the report format (i.e. HTML, PDF or CSV) and click the **Generate** button.

Token & SMS

- **Token Activities Report**
  This report lists down all token events/activities (e.g. GenerateOtp) based on the specified date &

time. To view this report, navigate to **Report Dashboard > Token & SMS > Historical** and click **Token Activities** on the left pane. Input the report criteria, select the report format (i.e. HTML, PDF or CSV) and the **Generate** button.

- **SMS OTP Usage Report**

  This report provides the number of SMS OTP usage per user ID. To view this report, navigate to **Report Dashboard > Token & SMS > Historical** and click **SMS OTP Usage** on the left pane. Input the report criteria, select the report format (i.e. HTML, PDF or CSV) and the **Generate** button.

Refer to *UAS Administration Guide - Report* and *AccessMatrix Common Administration Guide - Report Dashboard* for more details.

7. Appendices

A. SMS OTP Error Messages

This section describes some of the SMS OTP-related error messages.

Error Code	Error Class Name	Error Description	Solution
10020	Wrong Password	Wrong OTP. This will increment the bad login count.	Reenter the OTP sent to your mobile device or email within the validity time.
10402	DeniedByPolicyOtpReuseNotAllowed	Denied by the policy. OTP reuse is not allowed.	Check the **Allow reuse (verify) of OTP multiple times** attribute of the SMS OTP token policy.
10403	DeniedByPolicyMaxFailedAttemptsReached	Denied by the policy. Maximum failed attempts reached.	Check the **Lock Token If Bad Verification Count Is More Than** SMS OTP token policy attribute.
10404	DeniedByPolicyTokenExpired	Denied by the policy. Token has expired.	Check the **OTP validity (in seconds)** attribute of the SMS OTP token policy.

B. SMS OTP Logging

AccessMatrix utilizes Apache log4j module (that is a Java-based logging utility) as the logging framework. The log level can be configured via a property or XML file: **amlog4j2.properties**/**amlog4j2.xml**. This file is located in *<AccessMatrix_ROOT>/tomcat/webapps/am5/WEB-INF/classes/*.

You can configure this property for SMS OTP cache-related logging:

```
Properties (.properties files)
```

```
# SMS OTP Cache
log4j.logger.com.isprint.am.core.tokenmgmt.MemoryTokenStore=INFO
```

Log level can also be configured in Admin Console. Navigate to **Administration Dashboard > System > Logging** and click the **Edit** button. You can add a new log option or modify the existing log level. Save the changes.

> **Note**: Any modification here will not be persisted to the database, and all logging configurations will return to the default configured as defined in **amlog4j2.properties/amlog4j2.xml** once the AccessMatrix server is restarted.

Sample log showing SMS OTP is generated and delivered via HTTP Deliverer.

```
2019-01-24 14:22:37.583 [qtp993420850-120] INFO - (LocalEventManager.java:191) User
testuser2/NerlfJmDHh (real actor testuser2) sesId pEcl6Zi*** triggered event Token-GenerateOtp on
Token:MemoryTokenStore:oWQ3UImUnJ(SMS-oWQ3UImUnJ), txId:null, result:S, msg=GenerateOtp:
SMS-oWQ3UImUnJ
2019-01-24 14:22:37.584 [qtp993420850-120] WARN - (AuthenticationService.java:414)
com.isprint.am.dto.exception.AuthenticationException$ChallengeResponse
2019-01-24 14:22:37.585 [qtp993420850-120] INFO - (LocalEventManager.java:191) User
testuser2/NerlfJmDHh (real actor testuser2) sesId pEcl6Zi*** triggered event Authentication-Login, txId:null,
result:I, msg=ForceChgPwd: , code=18888, extCode=0
2019-01-24 14:22:37.586 [AM EventWorker-1-10] INFO - (DefaultEventNotifier.java:101) Found notification
policy with id=SMSOTP, but the event notification policy is disabled
2019-01-24 14:22:37.596 [AM EventWorker-1-11] DEBUG - (AuditWriter.java:240) Audit took 8ms
2019-01-24 14:22:37.618 [qtp993420850-120] DEBUG - (MethodLogger.java:28) XMLRPC:loginEx2 - exit,
lapse(ms)=175
2019-01-24 14:22:37.619 [qtp993420850-120] WARN - (XmlRpcAuthnService.java:150)
00001. credential done=3
00002. eventReported created=0
00003. Timeouts resolved=0
00004. Found realm=0
00005. lookup ok=3
00006. resp collected=137
00007. toTO.regenSesToken=0
00008. toTO.putAttrs=4
00009. exit * TOTAL *=147
2019-01-24 14:22:37.686 [AM EventWorker-1-10] INFO - (DefaultEventNotifier.java:163) Invoke notification
policy with id=SMSOTP User testuser2/NerlfJmDHh (real actor testuser2) sesId pEcl6Zi*** triggered event
Token-GenerateOtp on Token:MemoryTokenStore:oWQ3UImUnJ(SMS-oWQ3UImUnJ), txId:null, result:S,
msg=GenerateOtp: SMS-oWQ3UImUnJ, task=com.isprint.am.core.event.EventNotificationTask@56ae299f
2019-01-24 14:22:37.812 [AM EventWorker-1-10] INFO - (MessageDeliveryService.java:92) Message
queue and worker thread pool are created for delivery channel DefaultHTTPDeliveryChannel
(threadPool=java.util.concurrent.ThreadPoolExecutor@116492f1[Running, pool size = 0, active threads =
0, queued tasks = 0, completed tasks = 0]):
2019-01-24 14:22:37.812 [AM EventWorker-1-10] INFO - (MessageDeliveryService.java:93) +
maxThreads=15
2019-01-24 14:22:37.813 [AM EventWorker-1-10] INFO - (MessageDeliveryService.java:94) +
queueSize=80
2019-01-24 14:22:37.813 [AM EventWorker-1-10] INFO - (MessageDeliveryService.java:95) +
queueWaitTime (ms)=1000
2019-01-24 14:22:37.813 [AM EventWorker-1-10] INFO - (MessageDeliveryService.java:96) +
```

```
rejectedCountTimeWindow (mins)=60
2019-01-24 14:22:37.829 [AM EventWorker-1-10] INFO - (MessageDeliveryService.java:119) Message is queued for delivery channel DefaultHTTPDeliveryChannel, deliveryTicket=F0r0xkM3PlpIQWq3nR5rgg, elapsed=31 ms, source event=User testuser2/NerIfJmDHh (real actor testuser2) sesId pEcl6Zi*** triggered event Token-GenerateOtp on Token:MemoryTokenStore:oWQ3UImUnJ(SMS-oWQ3UImUnJ), txId:null, result:S, msg=GenerateOtp: SMS-oWQ3UImUnJ
2019-01-24 14:22:37.830 [AM DeliveryWorker-DefaultHTTPDeliveryChannel-1-1] INFO - (MessageDeliveryService.java:166) Processing for message delivery, deliivererId=DefaultHTTPDeliveryChannel, deliveryTicket=F0r0xkM3PlpIQWq3nR5rgg
2019-01-24 14:22:37.841 [AM EventWorker-1-10] DEBUG - (AuditWriter.java:240) Audit took 11ms
2019-01-24 14:22:38.604 [AM DeliveryWorker-DefaultHTTPDeliveryChannel-1-1] INFO - (HttpDeliverer.java:594) HttpDeliverer, message sent for +6583386106. Response resultCode = 200
```

C. Raw Http Delivery Channel

RawHttpDeliverer supports an SMS gateway that uses XML-based HTTP POST to deliver the message in the form of a short message service (SMS).

Configuration Tab

Attribute Name	Description
*Delivery Channel Id	Unique Id of delivery channel
*Delivery Channel Name	Name of delivery channel.
Description	Definition of delivery channel. (Optional)
Implementation	Type of delivery channel implementation, i.e. "RawHttpDeliverer". (Uneditable)
SMS Server Information	
*SMS Gateway Host	Refer to the hostname or IP Address of the SMS Gateway.
Method	Refer to the HTTP request method. Possible values: • "POST" • "GET" Default: POST
Request Content Type MIME type	Refer to the MIME type. Options: • "application/json" • "application/x-www-form-urlencoded" • "application/xml" • "application/soap+xml" • "text/html"

Attribute Name	Description
	• "text/xml" • "text/plain" Default: text/xml
Request Content Type Charset	Refers to the character encoding to use. Default: UTF-8
SMS Response Message Information	
HTTP Response Verification Way	Specify the method on how to detect the failure when sending a message. Options: • "HTTP Response Code" - based on the HTTP Response Code. • "HTTP Response Header" - based on the existence of a certain response header. • "HTTP Response Body" - based on the occurrence of a certain string in the response body. Default: HTTP Response Body
HTTP Response Status Header Name	The HTTP header name indicates the delivery status. Only required if "HTTP Response Header" is selected as **HTTP Response Verification Way**.
HTTP Response Status Header Value	The value of the HTTP Status Header. Only required if "HTTP Response Header" is selected as **HTTP Response Verification Way**.
Success Response Message	The text contained in the response body indicating successful sending. Only required if "HTTP Response Body" is selected as **HTTP Response Verification Way**. Example: "<hostReturnCode>0000</hostReturnCode>"
Request Headers	
Header to Attribute Mapping	Allows the setting of multiple HTTP headers, which must be defined in Custom Attributes. The custom attributes must not be part of the predefined attributes used by the system. Each entry must be in this format: <HTTP parameter name>=<AM's attribute name> (e.g. "Authorization=authzHeaderAttr", where "Authorization" is the HTTP header name and "authzHeaderAttr" is the attribute name).
Others	
TLS Version	Specify the TLS version(s) to use by HttpDeliverer for HTTPS connection. e.g. TLSv1, TLSv1.1, TLSv1.2 Default: TLSv1, TLSv1.1, TLSv1.2
Connection Timeout	The maximum amount of time waiting to connect to SMS Gateway Host. **Note:** Specified in milliseconds. Default: 0

Attribute Name	Description
Response Timeout	The maximum amount of time waiting to get a response from SMS Gateway Host. **Note:** Specified in milliseconds. Default: 0

D. Event Types

Event category	Event Type	Description
Authentication	GenerateOtp	The event when OTP is generated using Authentication Service.
	Login	The event when the user login.
Token	GenerateOtp	The event when OTP is generated using Token Service.
	VerifyOTP	The event when OTP is verified using Token Service.